

Atividade: Ambiente e Ferramentas de Desenvolvimento

! Important

Este roteiro é uma **atividade extra-classe**: siga os passos por conta própria antes de prosseguir com as práticas de bancada. Não é acompanhado ao vivo — se travar em algum passo, procure o professor pelos canais de atendimento do curso.

O desenvolvimento de *firmware* para sistemas embarcados exige a utilização de ferramentas específicas, responsáveis por viabilizar a criação, a compilação e a depuração do código. Nesta disciplina, evitaremos deliberadamente o uso de IDEs proprietárias — como o STM32CubeIDE — e, inclusive, de *plugins* de configuração automática, como o mais recente STM32 VS Code Extension. Essa escolha decorre da intenção de garantir que você não venha a depender de geradores de código automáticos, que tendem a esconder a complexidade real do hardware. Ao configurar seu próprio ambiente, passo a passo, você compreende exatamente como o *linker script*, os arquivos de inicialização e os *scripts* de depuração de fato funcionam, o que torna o conhecimento adquirido portátil para qualquer outra arquitetura ou fabricante.

Vamos instalar, nesta ordem:

- **WSL2** (Windows Subsystem for Linux 2) com **Ubuntu 24.04 LTS**;
- Acesso a dispositivos **USB** dentro do WSL2, necessário para gravar/depurar via **ST-LINK**;
- **GCC** e **Git**;
- **GNU Arm Embedded Toolchain** (`arm-none-eabi-gcc`);
- **OpenOCD** e as ferramentas **stlink** (gravação/depuração via SWD);
- **Visual Studio Code**, com as extensões necessárias.

i Note

Este roteiro assume Windows 10 (versão 2004/build 19041 ou superior) ou Windows 11 como sistema host. Se você já tem uma instalação Linux nativa (dual-boot) ou usa Mac, os passos de WSL2 não se aplicam — pule direto para a seção de instalação das ferramentas, que funciona igual em qualquer Linux.

Instalação do WSL2

O Windows 10 (a partir do *build* 19041) e o Windows 11 já trazem o comando `wsl` embutido; entretanto, o WSL **não vem habilitado por padrão** — é necessário ativá-lo explicitamente. É exatamente essa a função do comando `wsl --install`: ele habilita os recursos do Windows necessários (virtualização, núcleo Linux) e instala uma distribuição, tudo em uma única etapa — não existe, portanto, um passo separado de “ativar o WSL” que deva preceder esse comando.

Abra o **Windows PowerShell como administrador** e proceda à instalação do Ubuntu 24.04 LTS diretamente:

```
PS > wsl --install -d Ubuntu-24.04
```

Tip

Se quiser ver antes a lista completa de distribuições disponíveis, rode `wsl --list --online`. Usaremos especificamente o **Ubuntu 24.04 LTS** neste curso — se instalar uma distribuição diferente, você vai precisar adaptar alguns comandos deste roteiro por conta própria.

Ao final da instalação, será solicitada a escolha de um nome de usuário e de uma senha. **Atenção:** nenhum caractere aparece na tela durante a digitação da senha — trata-se de comportamento esperado; basta digitar e, em seguida, pressionar **Enter**.

Atualize a lista de pacotes e os programas já instalados (essa operação exige privilégio de administrador, razão pela qual se emprega o `sudo` — *super user do*):

```
foo@bar$ sudo apt update
foo@bar$ sudo apt upgrade
```

Crie os diretórios que serão utilizados ao longo de todo o curso: um destinado a arquivos baixados da internet, e outro que servirá como espaço de trabalho das atividades.

```
foo@bar$ cd
foo@bar$ mkdir Downloads
foo@bar$ mkdir sembl-workspace
```

Caso seja necessário acessar os arquivos do Linux a partir de um programa do Windows (ou simplesmente explorá-los graficamente), abra o Windows Explorer e digite `\\wsl$` na barra de endereços.

Acesso a dispositivos USB no WSL2 (ST-LINK)

O WSL não possui suporte nativo a dispositivos USB; para contornar essa limitação, emprega-se um sistema de compartilhamento por meio de rede IP, denominado **USB/IP**, que expõe o dispositivo USB do lado Windows (atuando como servidor) para o WSL (que atua como cliente).

Instalação do USB/IP

No lado **Windows**, instale o servidor USB/IP (`usbipd-win`):

```
PS > winget install usbipd
```

No lado **Ubuntu** (WSL), instale o cliente USB/IP, bem como o banco de dados de identificadores USB:

```
foo@bar$ sudo apt install linux-tools-generic hwddata
foo@bar$ sudo update-alternatives --install /usr/local/bin/usbip usbip /usr/lib/
linux-tools/*/usbip 20
```

Tip

Se o comando acima falhar informando que não encontrou o caminho, rode `ls /usr/lib/linux-tools/` para ver a versão exata instalada e ajuste o caminho no comando (`/usr/lib/linux-tools/<versão>/usbip`).

Ajustando permissões via udev

Antes de conectar o ST-LINK pela primeira vez, é necessário garantir que o seu usuário terá, de fato, permissão para utilizá-lo. Por padrão, um dispositivo USB recém-conectado pertence ao usuário `root`, o que impede o acesso por parte de um usuário comum. Trata-se do modelo de permissões característico do Linux: todo arquivo (inclusive os dispositivos, que no Linux também são representados como arquivos) possui um dono, um grupo e três conjuntos de permissões de leitura, escrita e execução (`rwX`) — aplicáveis, respectivamente, ao dono, ao grupo e a todos os demais usuários.

Para liberar o acesso ao ST-LINK a qualquer usuário, instalaremos regras `udev` — *scripts* que o Linux executa automaticamente sempre que um dispositivo é conectado. Os arquivos de regra correspondentes (`49-stlinkv1.rules`, `49-stlinkv2.rules`, `49-stlinkv2-1.rules`, `49-stlinkv3.rules`) encontram-se disponíveis no repositório público da disciplina (`ufu-sembl-src/tools/udev/rules.d`). Baixe-os diretamente pelo terminal e copie-os para `/etc/udev/rules.d`:

```
foo@bar$ cd ~/Downloads
foo@bar$ for f in 49-stlinkv1 49-stlinkv2 49-stlinkv2-1 49-stlinkv3; do
wget https://raw.githubusercontent.com/daniel-p-carvalho/ufu-sembl-src/master/
tools/udev/rules.d/$f.rules; done
foo@bar$ sudo cp 49-stlink*.rules /etc/udev/rules.d/
```

(Caso o repositório `ufu-sembl-src` já tenha sido clonado, basta copiar diretamente os arquivos presentes na pasta `tools/udev/rules.d`).

Reinicie o serviço `udev` e solicite a ele que recarregue as regras:

```
foo@bar$ sudo service udev restart
foo@bar$ sudo udevadm control --reload
```

A partir deste ponto, toda vez que um ST-LINK for conectado ao Ubuntu, ele já aparecerá com permissão de leitura e escrita para qualquer usuário.

Conectando o gravador ST-LINK

Conecte o **ST-LINK** à porta USB do computador. Conectá-lo ao WSL envolve dois passos, ambos executados no PowerShell do Windows: **compartilhar** o dispositivo (**bind**), procedimento realizado uma única vez, e **anexá-lo** ao WSL (**attach**), procedimento repetido a cada sessão de trabalho.

Em primeiro lugar, liste os dispositivos USB reconhecidos pelo Windows:

```
PS > usbipd list
```

Anote o **BUSID** do ST-LINK (algo como **3-3** — valor que pode variar conforme a porta USB utilizada). A coluna **STATE** indica se o dispositivo está **Not shared** (não compartilhado), **Shared** (compartilhado, mas ainda não anexado) ou **Attached** (já anexado ao WSL).

Compartilhar (uma única vez). Abra o PowerShell **como administrador** e proceda ao compartilhamento do ST-LINK:

```
PS > usbipd bind --busid 3-3
```

Esse é o único passo que efetivamente exige privilégio de administrador. O compartilhamento permanece registrado no Windows, valendo mesmo após a desconexão do ST-LINK ou a reinicialização do computador, desde que ele seja conectado à mesma porta USB. Ao trocar de porta, o BUSID se altera, sendo necessário repetir o **bind** para o novo valor.

Anexar ao WSL (a cada sessão). Com o Ubuntu já aberto, anexe o ST-LINK ao WSL — em um PowerShell comum, sem privilégio de administrador:

```
PS > usbipd attach --wsl --busid 3-3
```

No terminal do Ubuntu, verifique se o dispositivo está visível:

```
foo@bar$ lsusb
```

Uma das linhas exibidas deve corresponder ao ST-LINK, algo como **Bus 001 Device 002: ID 0483:3748 STMicroelectronics ST-LINK/V2**. Os números de *Bus* e *Device* indicam o arquivo que representa o dispositivo. Verifique que ele possui permissão de leitura e escrita para todos os usuários (**crw-rw-rw-**), graças às regras **udev** instaladas no passo anterior:

```
foo@bar$ ls -l /dev/bus/usb/001/002
```

💡 Tip

O `attach` precisa ser feito sempre que o ST-LINK for desconectado fisicamente, o computador for reiniciado ou o WSL for encerrado. Se preferir que o Windows anexe o ST-LINK automaticamente sempre que ele for conectado, use `usbipd attach --wsl --busid 3-3 --auto-attach` e deixe esse PowerShell aberto.

Desanexar. Enquanto o ST-LINK permanecer anexado ao WSL, ele ficará **indisponível para o Windows**. Para devolvê-lo ao lado Windows — por exemplo, a fim de utilizar uma ferramenta que roda nativamente ali, como o STM32CubeProgrammer —, proceda ao desanexamento:

```
PS > usbipd detach --busid 3-3
```

Um `detach` seguido de um novo `attach` constitui também a forma mais rápida de “reconectar” o ST-LINK nas situações em que o `st-info` ou o OpenOCD deixam de encontrá-lo.

Compilador, Make e controle de versão

Instale o **GCC** (compilador C destinado à sua própria máquina, empregado apenas para testes rápidos), o **GNU Make** e o **Git**:

```
foo@bar$ sudo apt install build-essential git
```

O pacote `build-essential` já traz consigo o **GNU Make**. A automação do processo de *build* ao longo do curso é realizada com o Make: o `Makefile` é escrito manualmente, regra por regra, em vez de se recorrer a geradores como o **CMake**. Esse procedimento garante a compreensão exata de cada etapa do processo — compilação, geração de dependências, *linking* e conversão para binário —, sem que nenhuma camada adicional esconda o que, de fato, está ocorrendo. Ferramentas de *build* de nível mais elevado, como o CMake, poderão ser exploradas mais adiante, já no contexto do projeto integrador, quando a escala do projeto vier a justificar essa camada extra de automação.

Configure o seu nome e o seu e-mail no Git — o comando `git clone` será utilizado para distribuir os arquivos de cada atividade:

```
foo@bar$ git config --global user.name "seu nome aqui"
foo@bar$ git config --global user.email "seu e-mail aqui"
```

GNU Arm Embedded Toolchain

O `arm-none-eabi-gcc` corresponde ao compilador **cruzado**: executa-se na própria máquina do usuário (x86_64), mas gera código destinado à arquitetura ARM Cortex-M. O projeto é hospedado no GitLab da ARM:

Arm GNU Toolchain — releases → escolha a *tag* da versão mais recente (no momento em que este roteiro foi escrito, **15.3.rel1**) → dentro da página da release, copie o link do pacote `.tar.xz` para Linux x86_64 (`arm-none-eabi`).

 Tip

Versão atual usada neste roteiro: **15.3.rel1**. Confira em gitlab.arm.com/tooling/gnu-toolchains-for-arm se já existe uma versão mais nova e ajuste o número abaixo antes de baixar.

```
foo@bar$ cd ~/Downloads
foo@bar$ wget https://gitlab.arm.com/api/v4/projects/tooling%2Fgnu-toolchains-for-arm/packages/generic/gnu-toolchain/15.3.rel1/arm-gnu-toolchain-15.3.rel1-x86_64-arm-none-eabi.tar.xz
```

Descompacte o pacote no diretório `/usr/share` e crie *links* simbólicos em `/usr/bin`, de modo a facilitar a utilização:

```
foo@bar$ sudo tar xf arm-gnu-toolchain-15.3.rel1-x86_64-arm-none-eabi.tar.xz -C /usr/share/
foo@bar$ sudo ln -s /usr/share/arm-gnu-toolchain-15.3.rel1-x86_64-arm-none-eabi/bin/* /usr/bin/
```

Verifique a instalação:

```
foo@bar$ arm-none-eabi-gcc --version
foo@bar$ arm-none-eabi-g++ --version
foo@bar$ arm-none-eabi-gdb --version
```

OpenOCD

Tente, em primeiro lugar, instalar via `apt` — versões recentes do Ubuntu já trazem um OpenOCD razoavelmente atualizado, com bom suporte a STM32/ST-LINK:

```
foo@bar$ sudo apt install openocd
foo@bar$ openocd --version
```

Tip

Caso a versão instalada seja antiga (inferior a 0.12) ou não reconheça o gravador utilizado, compile-a a partir do código-fonte: clone <https://git.code.sf.net/p/openocd/code>, verifique as *tags* disponíveis com `git tag`, mude para a última *tag* estável (`git switch -c <tag>`) e siga o processo `./bootstrap && ./configure --enable-stlink && make && sudo make install`, descrito em detalhe no [README](#) do próprio repositório.

Ferramentas stlink

As ferramentas **st-info**, **st-flash** e **st-util** (esta última utilizada pelo VS Code por meio do Cortex-Debug) complementam o OpenOCD. Tente, primeiramente, instalá-las via `apt`:

```
foo@bar$ sudo apt install stlink-tools
foo@bar$ st-info --version
```

Tip

Caso seja necessária uma versão mais recente, compile-a a partir do repositório oficial `stlink-org/stlink`: clone o repositório, verifique as *tags* com `git tag`, mude para a última *tag* estável e execute `make release && sudo make install` (instalando previamente as dependências `cmake libusb-1.0-0 libusb-1.0-0-dev`).

Visual Studio Code

Baixe e instale o **VS Code para Windows** em code.visualstudio.com/download.

Instale as extensões a seguir (**Ctrl+Shift+X** abre a loja de extensões):

- **Remote - WSL** — permite que o VS Code rodando no Windows edite e execute comandos diretamente dentro do WSL;
- **C/C++** (ou o **C/C++ Extension Pack**) — edição, IntelliSense e depuração em C/C++;
- **Cortex-Debug** — depuração de dispositivos ARM Cortex-M via SWD/JTAG, integrada ao OpenOCD/GDB.

Após instalar a extensão **Remote - WSL**, clique no ícone verde situado no canto inferior esquerdo do VS Code e abra uma nova janela remota conectada ao WSL. É importante observar que extensões instaladas localmente no Windows não valem automaticamente dentro do WSL — sendo necessário instalar novamente **C/C++** e **Cortex-Debug** no interior dessa janela remota.

Verificação final

Ao final deste roteiro, os comandos a seguir devem funcionar sem erro dentro do terminal Ubuntu (WSL):

```
foo@bar$ arm-none-eabi-gcc --version
foo@bar$ openocd --version
foo@bar$ st-info --version
foo@bar$ git --version
```

E o VS Code (na janela remota WSL) deve exibir **Remote - WSL**, **C/C++** e **Cortex-Debug** na lista de extensões instaladas.

Validação com a placa

Os comandos anteriores comprovam que as ferramentas foram **instaladas**, mas não que elas **funcionam em conjunto**: que o USB efetivamente chega ao WSL, que as regras `udev` estão corretas, que o ST-LINK consegue se comunicar com o microcontrolador e que o depurador é capaz de controlá-lo. A única forma de verificar isso é utilizar a cadeia completa. Nesta última etapa, você compilará um programa, o gravará na placa e o executará passo a passo.

O programa em questão é um *blink*: faz o LED da placa piscar. Não se preocupe, neste momento, em compreender o código — ao longo dos próximos capítulos, cada linha dele será estudada em detalhe, e as suas peças serão reescritas a partir do zero.

Conectando o ST-LINK à Blackpill

Ligue o ST-LINK à Blackpill por meio dos quatro pinos do conector SWD, situado na lateral da placa:

ST-LINK	Blackpill
SWDIO	DIO (PA13)
SWCLK	SCK (PA14)
GND	GND
3.3V	3V3

Warning

Os clones de ST-LINK nem sempre seguem a mesma ordem de pinos — confira a serigrafia do **seu** gravador antes de ligar. E não alimente a placa pelo USB-C e pelo **3.3V** do ST-LINK ao mesmo tempo.

Conecte o ST-LINK ao computador, anexe-o ao WSL (`usbipd attach --wsl --busid <BUSID>` no PowerShell — o `bind` já foi realizado na seção sobre USB) e confirme que o chip é devidamente reconhecido:

```
foo@bar$ st-info --probe
```


A saída deve exibir o identificador `chipid: 0x0431` (ou `0x431`), correspondente à família STM32F411, acompanhado de `flash: 524288` (512 KB) e `sram: 131072` (128 KB). Caso o `st-info` não encontre nada, revise a fiação e as regras `udev`.

Compilando e gravando

Os códigos-fonte e os roteiros práticos da disciplina encontram-se centralizados no repositório oficial `ufu-sembl-src`, organizados em uma pasta para cada laboratório (`lab-01`, `lab-02` etc.) e uma pasta de ferramentas de apoio (`tools/`). Os laboratórios que rodam na placa possuem, em seu interior, uma pasta correspondente a cada placa suportada; a da placa oficial é a `stm32f411-blackpill`. Caso outra placa esteja sendo utilizada, consulte o `README.md` do laboratório correspondente. O clone desse repositório deve ser mantido no espaço de trabalho como referência de consulta e ponto de partida para as práticas, devendo ser atualizado com `git pull` ao longo de todo o curso.

Para este primeiro teste, clone o repositório e acesse a pasta do **Lab 01** correspondente à nossa placa, que contém o programa `blink` completo para a Blackpill:

```
foo@bar$ cd ~/sembl-workspace
foo@bar$ git clone https://github.com/daniel-p-carvalho/ufu-sembl-src.git
foo@bar$ cd ufu-sembl-src/lab-01/stm32f411-blackpill
```

Compile o projeto e grave-o na placa:

```
foo@bar$ make clean && make
foo@bar$ make flash
```

O `make` utiliza o `arm-none-eabi-gcc` para gerar o executável `blink.elf` e a imagem `blink.bin`. O `make flash`, por sua vez, utiliza o `st-flash` para gravar essa imagem na Flash do microcontrolador, a partir do endereço `0x08000000`, e reinicia o chip em seguida. **O LED azul da placa deve começar a piscar.** Caso isso ocorra, o `toolchain`, o acesso USB, as permissões e as ferramentas do ST-LINK estão funcionando plenamente.

Executando passo a passo no VS Code

Resta ainda validar o depurador. No VS Code (na janela remota conectada ao WSL), abra a pasta `stm32f411-blackpill` do Lab 01 por meio do menu **File** → **Open Folder** (`~/sembl-workspace/ufu-sembl-src/lab-01/stm32f411-blackpill`). É imprescindível que seja essa pasta específica, e não `lab-01`: o VS Code busca a configuração exatamente na pasta que foi aberta.

Para que a extensão **Cortex-Debug** saiba como inicializar o OpenOCD e depurar o microcontrolador, o projeto necessita de instruções contidas no arquivo `.vscode/launch.json`. Seguindo a mesma lógica centralizada já empregada nas regras do `udev` (que residem em `tools/udev/`), disponibilizamos modelos padronizados de configuração do editor na pasta `tools/vscode/` do repositório:

- `launch.json`: configuração do servidor OpenOCD, ponteiro para o arquivo ELF e parada no `main()`;
- `tasks.json`: comandos de compilação integrados ao atalho `Ctrl+Shift+B`;
- `c_cpp_properties.json`: definição do compilador cruzado para o IntelliSense, evitando falsos avisos de erro em tipos de dados de sistemas embarcados.

O Lab 01 é o único que já traz uma cópia pronta desses arquivos, na pasta oculta `.vscode/`, justamente para que esta validação funcione sem qualquer ajuste adicional. Nos laboratórios subsequentes, será necessário copiar os modelos para a pasta do projeto e adaptá-los; o `README.md` de `tools/vscode/` explica detalhadamente o que se altera em cada arquivo:

```
foo@bar$ cp -r ~/sembl-workspace/ufu-sembl-src/tools/vscode .vscode
```

O conteúdo do `launch.json` empregado pelo Cortex-Debug é bastante direto:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Blinky - STM32F411 Blackpill (OpenOCD)",
      "type": "cortex-debug",
      "request": "launch",
      "serverType": "openocd",
      "cwd": "${workspaceFolder}",
      "executable": "${workspaceFolder}/blinky.elf",
      "configFiles": [
        "interface/stlink.cfg",
        "target/stm32f4x.cfg"
      ],
      "runToEntryPoint": "main"
    }
  ]
}
```

1. Abra `src/main.c` e posicione um *breakpoint* (clitando à esquerda do número da linha) nas duas linhas que escrevem em `*gpio_c_bsr`, no interior do `while (1)`: uma imediatamente abaixo do comentário “Liga o LED”, e outra abaixo de “Desliga o LED”.
2. Pressione **F5** para iniciar a depuração. A execução é interrompida na primeira linha do `main()`.
3. Pressione **F5** (*Continue*) novamente: a execução para no primeiro *breakpoint*, **antes** da escrita no registrador.
4. Pressione **F10** (*Step Over*) para executar essa linha. O LED muda de estado.
5. Continue alternando entre **F5** e **F10**, observando o LED acender e apagar sob seu controle direto. Aproveite também para abrir o painel **Cortex Registers** e acompanhar os registradores do processador se alterando a cada passo.

 Tip

Evite utilizar o *Step Over* sobre os laços `for (i = 0; i < LED_DELAY; i++)`. Esses laços de atraso possuem 100 mil iterações, e o depurador as executaria uma instrução por vez, via SWD — o que poderia levar minutos e dar a impressão de travamento. É justamente por essa razão que se emprega *breakpoints* seguidos de *Continue*: entre um *breakpoint* e outro, o processador executa em velocidade normal.

Para encerrar, interrompa a depuração (**Shift+F5**). O programa permanece gravado na Flash e volta a piscar sempre que a placa for ligada.

i Alternativa: depurando pelo terminal com o GDB

Caso seja de sua preferência, ou caso o Cortex-Debug apresente algum problema, a mesma sessão pode ser realizada diretamente pelo terminal. Em um primeiro terminal, inicie o OpenOCD e deixe-o em execução:

```
foo@bar$ make openocd
```

Em um segundo terminal, na mesma pasta, conecte o GDB (o comando `make debug` já interrompe o processador no reset):

```
foo@bar$ make debug
(gdb) break main
(gdb) continue
```

O GDB interrompe no início de `main()`. Para controlar o LED passo a passo, posicione *breakpoints* nas duas escritas em `*gpio_cbsrr` — as linhas que ligam e desligam o LED. O número exato da linha pode variar se o código for editado; para descobri-lo sem abrir o arquivo, liste o trecho relevante diretamente no GDB:

```
(gdb) list main,main+60
```

Identifique as duas atribuições a `*gpio_cbsrr` e crie os *breakpoints* com o número de linha correspondente. Por exemplo, se as escritas estiverem nas linhas 157 e 163:

```
(gdb) break main.c:157
(gdb) break main.c:163
(gdb) continue
(gdb) next
(gdb) continue
(gdb) next
(gdb) info registers
(gdb) quit
```

Cada `continue` executa até o próximo *breakpoint*, ao passo que cada `next` executa a escrita correspondente e altera o estado do LED.

Configurando o Repositório de Entregas

Ao longo do curso, você desenvolverá seus laboratórios e responderá aos estudos dirigidos dentro do diretório `~/seml-workspace`. Para organizar as entregas e manter um backup seguro do seu trabalho, transformaremos esse espaço em um repositório Git privado no GitHub.

Execute os passos a seguir rigorosamente na ordem apresentada:

1. Identifique-se para o Git. Cada *commit* registra o nome e o endereço de e-mail do respectivo autor. Utilize o seu **e-mail institucional** (@ufu.br), garantindo que ele também esteja adicionado à sua conta do GitHub:

```
foo@bar$ git config --global user.name "Seu Nome Completo"
foo@bar$ git config --global user.email "seu.nome@ufu.br"
foo@bar$ git config --global init.defaultBranch master
```

2. Crie o repositório no GitHub. Acesse a sua conta do GitHub (crie uma, caso ainda não possua) e proceda à criação de um repositório novo:

- nome: `seml-entregas`;
- visibilidade: **Private**;
- **não** marque as opções de criação de `README`, `.gitignore` ou licença: o repositório precisa ser criado completamente vazio.

3. Conceda acesso ao professor. No repositório recém-criado, acesse *Settings* → *Collaborators* → *Add people* e convide o usuário `daniel-p-carvalho`. Na ausência desse convite, o professor não terá como visualizar as suas entregas semanais.

4. Crie uma chave SSH. O GitHub não aceita a sua senha da conta no momento do envio de código. A forma mais prática e segura de autenticação consiste em uma chave SSH, criada uma única vez. Aceite o local padrão sugerido para o arquivo pressionando **Enter**:

```
foo@bar$ ssh-keygen -t ed25519 -C "seu.nome@ufu.br"
foo@bar$ cat ~/.ssh/id_ed25519.pub
```

O segundo comando exibe a sua chave **pública**, uma linha que se inicia por `ssh-ed25519`. Copie a linha em sua totalidade e cadastre-a no GitHub, em *Settings* (referente à sua conta, e não ao repositório) → *SSH and GPG keys* → *New SSH key*. Em seguida, teste a conexão:

```
foo@bar$ ssh -T git@github.com
```

Na primeira tentativa, o SSH indaga se o servidor deve ser considerado confiável. Responda **yes**. A resposta esperada inicia-se por `Hi <seu-usuário>! You've successfully authenticated`. O arquivo `~/.ssh/id_ed25519` (sem a extensão `.pub`) corresponde à sua chave **privada**: ela jamais deve ser copiada ou enviada a terceiros.

5. Transforme o espaço de trabalho em repositório.

```
foo@bar$ cd ~/seml-workspace
foo@bar$ git init
```

6. Informe ao Git o que não deve ser versionado. Crie, com o editor de sua preferência (`nano` ou VS Code), o arquivo `~/seml-workspace/.gitignore`, com o seguinte conteúdo:

```
# O clone do repositório da disciplina não é seu
ufu-sembl-src/

# Arquivos gerados pela compilação
*.o
*.i
*.s
*.elf
*.bin
*.map
build/
```

7. Faça o primeiro *commit*. O comando `git add .` prepara todos os arquivos não ignorados do diretório atual.

```
foo@bar$ git status
foo@bar$ git add .
foo@bar$ git commit -m "Configuracao inicial e lab-01"
```

8. Associe o repositório local ao remoto e efetue o envio. Substitua `<seu-usuário>` pelo seu nome de usuário no GitHub:

```
foo@bar$ git remote add origin git@github.com:<seu-usuário>/sembl-entregas.git
foo@bar$ git push -u origin master
```

Abra o repositório no navegador e verifique que a pasta `lab-01` e o arquivo `.gitignore` encontram-se presentes, e que a pasta `ufu-sembl-src` foi devidamente ignorada.

i Note

A partir deste ponto, o ciclo de entrega será sempre o mesmo para todas as semanas: trabalhe nos seus arquivos, verifique as alterações com `git status`, prepare com `git add`, registre com `git commit -m "Sua mensagem"` e envie com `git push`. A opção `-u`, empregada no primeiro `push`, associou o seu ramo local ao remoto, de modo que os envios subsequentes exigem apenas `git push`.

! Entrega

1. **Repositório:** certifique-se de que a estrutura do seu `lab-01` foi enviada com sucesso para o seu repositório `seml-entregas` no GitHub, e de que o professor foi convidado como colaborador.
2. **Demonstração:** grave um vídeo curto (de até um minuto) demonstrando: a saída do `st-info --probe` no terminal, o LED piscando na placa após o `make flash`, e uma sessão de depuração na qual o LED é acendido e apagado passo a passo pelo GDB (Cortex-Debug ou terminal). Envie o link do vídeo pelo canal indicado pelo professor.

Com isso, o ambiente encontra-se validado de ponta a ponta e o seu sistema de controle de versão está configurado, deixando o *workspace* devidamente preparado para as práticas subsequentes do curso.

Referências

- [1] Instalar o WSL
- [2] Conectar dispositivos USB ao WSL (usbipd-win)
- [3] Arm GNU Toolchain — releases (GitLab)
- [4] OpenOCD
- [5] stlink-org/stlink
- [6] STM32F411xC/E Datasheet
- [7] RM0383 Reference Manual — STM32F411xC/E advanced ARM-based 32-bit MCUs
- [8] Using the GNU Compiler Collection (GCC)
- [9] GNU Make Manual
- [10] Using ld, the GNU linker
- [11] Chacon, S.; Straub, B. *Pro Git*, 2ª edição. Apress, 2014. Tradução para o português disponível gratuitamente. Capítulos 1 e 2.
- [12] Documentação do GitHub — criação de repositórios, colaboradores e conexão com o GitHub por SSH.